Understanding fork(), PID, wait(), and exit()

A beginner-friendly C program for Linux / POSIX systems

1. Complete C program

This example creates a child process, prints process IDs, waits for that specific child, and reads the child's exit status. It uses **waitpid()** so the parent waits for the child it created.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid;

    printf("Program started. My PID = %ld\n", (long)getpid());
    fflush(stdout);

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(EXIT_FAILURE);
    }
    else if (pid == 0)
    {
        printf("\nI am the CHILD process\n");
        printf("Child PID  = %ld\n", (long)getpid());
        printf("Parent PID = %ld\n", (long)getppid());
        printf("Child is doing its work...\n");
        exit(5);
    }
    else
    {
        int status;
        pid_t finished_pid = waitpid(pid, &status, 0);

        if (finished_pid == -1)
        {
            perror("waitpid failed");
            exit(EXIT_FAILURE);
        }

        printf("\nI am the PARENT process\n");
        printf("Parent PID = %ld\n", (long)getpid());
        printf("Child PID  = %ld\n", (long)pid);
        printf("Parent has collected the child's status.\n");

        if (WIFEXITED(status))
            printf("Child exited with status = %d\n", WEXITSTATUS(status));

        printf("Parent continues after child finishes.\n");
    }

    return 0;
}
```

2. What each function does

fork(): Creates a child process. It returns a negative value on failure, 0 in the child, and the child's PID in the parent.

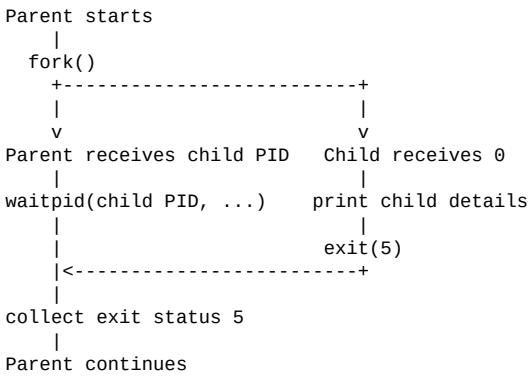
getpid(): Returns the PID of the process that calls it.

getppid(): Returns the parent PID of the calling process.

waitpid(pid, &status, 0): Waits for the specified child to finish and collects its termination status, allowing the parent to reap the child.

exit(5): Terminates the child with exit status 5. The parent reads it with **WEXITSTATUS(status)** after confirming **WIFEXITED(status)**.

3. Process flow



4. Build and run

```
gcc fork_demo.c -o fork_demo
./fork_demo
```

Save the program as **fork_demo.c**. Run these commands in a Linux terminal or a POSIX-compatible environment such as WSL. Native Windows compilers generally do not provide fork().

5. Example output (PIDs will differ)

```
Program started. My PID = 4100

I am the CHILD process
Child PID  = 4101
Parent PID = 4100
Child is doing its work...

I am the PARENT process
Parent PID = 4100
Child PID  = 4101
Parent has collected the child's status.
Child exited with status = 5
Parent continues after child finishes.
```

Output ordering can vary because the operating system schedules processes. The parent section shown after waitpid() runs only after that child has finished; messages printed before the wait may appear in different orders in other examples.

6. Quick knowledge check

Question: What does fork() return in the child process?
Answer: 0. In the parent it returns the child's PID; on failure it returns a negative value.

Remember: fork() creates the child; a PID identifies a process; waitpid() waits and collects status; exit() terminates a process and supplies an exit status.